Universidad de Chile
Facultad de Ciencias Físicas y Matemáticas
Departamento de Ciencias de la Computación

Static Load Balancing for a CPU + GPU Stencil

Computación en GPU

Author: Matías Saavedra

Document: engineering report (hybrid load balancing)
Last updated: Tuesday 14th July, 2026
Santiago de Chile

Resumen

The `HybridEngine` runs several compute nodes on one Game of Life board (HighLife, `B36/S23`), split by rows with Divisible Load Theory (DLT, Barlas $\mathcal{S}11.3$). This report analyses static load balancing on a GTX 1660 Ti Max-Q, first paired with a 12-thread CPU and then with the machine's Intel UHD 630 integrated GPU. The CPU+GPU pair is a textbook "one node dominates" regime: the GPU is $\sim$160$\times$ the CPU, so the DLT-optimal CPU share is 0.6 % and the balancing ceiling is smaller than the per-step coordination cost — the hybrid does not beat pure GPU, and a raw sweep that appears to is a kernel confound (the hybrid runs a faster, branch-free halo kernel). The apparent "OpenCL regression" seen during that run was a misdiagnosis: the slow 1.7 Gcells/s figure is the integrated GPU, which the default OpenCL selection picks as platform 0, not a driver fault. Reusing that idle iGPU as a genuine second DLT node changes the verdict. Because the inter-node seam is a single fixed row while compute grows as N^2, the communication cost vanishes as $1/N$: a well-chosen static iGPU + dGPU split beats pure dGPU by a margin that grows with the grid, reaching +5.5 % at $N = 32768$ and 99 % of the $R_{\text{iGPU}} + R_{\text{dGPU}}$ ceiling. The prize predicted by DLT is real once the load is large enough to amortize the seam — though the default auto-calibration still overshoots the optimum.

Índice de Contenidos

Índice de Figuras

Índice de Tablas

1. Scope and setup

Backend under test: the `HybridEngine` (branch `static_load_balancing`) — CPU + GPU on one board, split by rows, the partition chosen by DLT. Hardware: an NVIDIA GTX 1660 Ti Max-Q (`sm_75`, 60 W power ceiling) and a 12-thread CPU, on Linux. Data: `results/linux/sweep_{gpu_c` from `scripts/sweep.sh`; figure `img/hybrid_load_balancing.png`.

- Metric. Cells evaluated per second, Mcells/s = $(\text{rows} \times \text{cols} \times \text{gens})/(t \times 10^6)$, measured with the `NullRenderer` so host $\leftrightarrow$ device transfers and drawing never enter the timed loop. Timing is in-program and uniform per backend (CPU and hybrid: `std::chrono`; CUDA: `cudaEvent`). The hybrid figure is the wall of the concurrent CPU+GPU region, i.e. $\approx$ the slower of the two nodes.
- Sweep. Experiment A is the GPU block $\times$ shared/local sweep; experiment B is the scaling vs. N for CPU(threads) / CUDA / OpenCL / hybrid. We report the best (highest-Mcells) row per configuration, matching "mejor tiempo" in the graded report. Bounded edges, deterministic random fill (`-seed 1`: identical work across backends).
- Controlled follow-ups (this analysis). Single-knob runs that isolate why the hybrid behaves as it does: pure CUDA vs. the halo kernel on the full board (`-cpu-frac 0`) vs. the auto-split hybrid, all at block 128 so the only variable is the code path.

GPU health was verified before and after the sweep (idle 4.9 W, 39–41 °C, 60 W ceiling, no power-cap creep, no throttle flags), so the GPU numbers are not thermally compromised. Run-to-run boost-clock variance is real, though — see $\mathcal{S}$5.4.

2. Throughput results

Tabla 1: Best throughput per backend per grid size N (Gcells/s, kernel time). The last column, "GPU halo-only", is the hybrid's GPU path run with `-cpu-frac 0` (halo kernel, full board, no CPU) — a control that isolates the kernel from the load balancing.

N	CPU (best t)	CUDA	OpenCL	Hybrid (auto)	GPU halo-only
512	0.151	24.76	1.67	5.20	14.57
1024	0.146	27.55	1.70	9.56	25.07
2048	0.144	31.52	1.70	19.91	30.70
4096	0.142	32.25	1.69	22.75	31.42
8192	0.147	26.20	1.69	25.12	28.79
16384	0.152	24.83	1.68	28.46	28.72

The headlines from the table:

- The CPU plateaus at ∼0.15 Gcells/s regardless of N: a 1-byte 9-point stencil is memory-bandwidth bound, and beyond ∼6 threads the cores contend for the same bandwidth (scaling is flat, not linear). This sets R_{cpu}.
- CUDA is ∼160–230× the CPU, peaking ∼32 Gcells/s at $N \approx 2048$–4096, then easing to ∼25 at $N = 16384$ (larger working sets spill L2, so the kernel becomes more memory-bound).
- The "OpenCL" column reads 1.7 Gcells/s, ∼15× below CUDA. This is not a slow NVIDIA path but the Intel UHD 630 integrated GPU, which the default OpenCL selection picks as platform 0 (see $\mathcal{S}$5.3); pinned to NVIDIA, OpenCL matches CUDA. That idle iGPU becomes the second node of the two-GPU study ($\mathcal{S}$6).

3. The load-balancing split (DLT $\mathcal{S}$11.3)

The calibration phase times a few CPU-only and GPU-only steps on the actual grid and applies the all-finish-together optimum:

$$\text{part}_{\text{gpu}} = \frac{R_{\text{gpu}}}{R_{\text{cpu}} + R_{\text{gpu}}}, \qquad \text{part}_{\text{cpu}} = 1 - \text{part}_{\text{gpu}}, \tag{1}$$

with $R = 1/p$ the measured throughput (cells/s), converted to a row split $s = \text{round}(\text{part}_{\text{cpu}} \cdot R)$.

Tabla 2: Calibrated rates and the resulting split. R in Mcells/s. The split is stable at 0.6 % CPU at every size, exactly as $R_{\text{cpu}}/R_{\text{gpu}} \approx 0.006$ predicts.

N	R_{cpu}	R_{gpu}	GPU/CPU	CPU rows	CPU share	DLT bound $R_{\text{cpu}}+R_{\text{gpu}}$
2048	157.6	24 504	156×	13	0.6 %	24 661
4096	163.8	27 176	166×	25	0.6 %	27 340
8192	158.5	25 271	159×	51	0.6 %	25 430
16384	158.0	27 286	173×	94	0.6 %	27 444

The split matches the $\mathcal{S}$11.3 optimum to the row, and the two nodes finish together: at $N = 16384$ the CPU needs $94 \times 16384/158$ Mcells/s ≈ 9.74 ms and the GPU needs $16\,290 \times 16384/27\,286$ Mcells/s ≈ 9.78 ms. The DLT machinery works precisely as the book describes — this is not a tuning failure.

The catch is the ceiling. Because the CPU's optimal share is only 0.6 %, the DLT "equivalent node" throughput $R_{\text{cpu}} + R_{\text{gpu}}$ is at most 0.6 % above the GPU alone. That is the entire prize, and it is tiny.

4. The figure

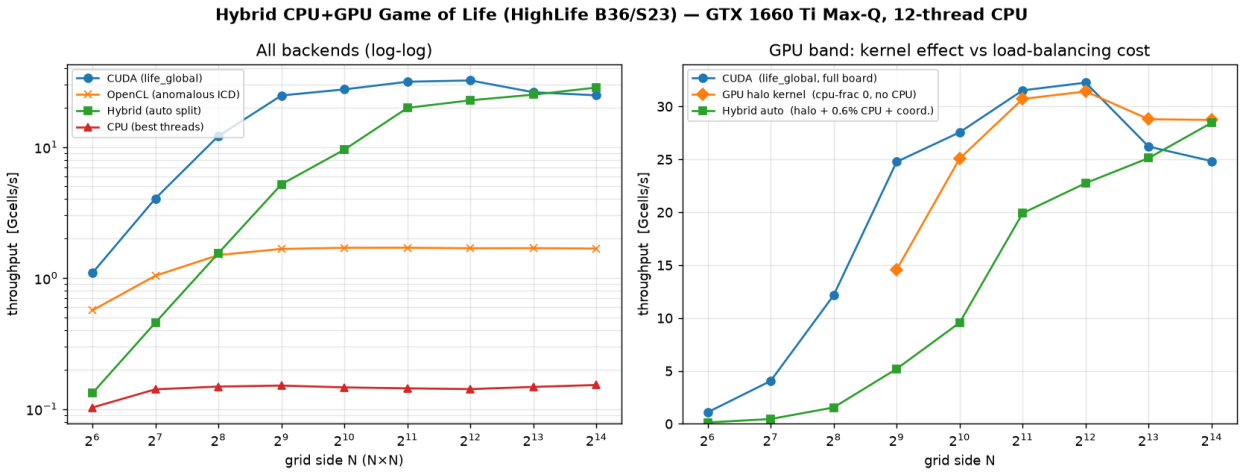

Figura 1: Hybrid scaling. Left (log–log, all backends): the GPU backends sit ∼160–230× above the flat CPU line; the OpenCL curve at ∼1.7 Gcells/s is the Intel iGPU (the default OpenCL device), not a slow NVIDIA path. Right (GPU band): CUDA (`life_global`) peaks early then sags; the halo kernel holds ∼29 Gcells/s at large N; the auto-split hybrid climbs from far below toward the GPU band as the per-step overhead amortizes, only catching up near $N = 16384$.

5. Unexpected findings

5.1. "Hybrid beats CUDA at large N" is a kernel effect, not load balancing

The raw sweep shows the hybrid mean exceeding the CUDA mean at $N = 16384$ (27.4 vs. 24.6 Gcells/s, +11 %) with low variance — too consistent to be noise, yet impossible to attribute to the CPU, whose DLT-optimal contribution is only 0.6 %. A controlled run resolves it (block 128, full board):

Tabla 3: Like-for-like at block 128, mean of repeated runs (Gcells/s). The halo kernel beats `life_global` with zero CPU; adding the CPU and the coordination (hybrid auto) gives it all back.

N	CUDA `life_global`	Halo kernel (`--cpu-frac 0`, no CPU)	Hybrid (auto)
4096	25.68	26.61 (+3.6 %)	18.25 (−29 %)
16384	25.14	28.12 (+12 %)	25.39 (+1 %)

The halo kernel is intrinsically faster than `life_global`, with zero CPU involvement. The reason: `life_global` does a per-cell vertical bounds check (`if (nr < 0 || nr >= rows)`, or a modulo under `-wrap`) for every neighbour row; the halo kernel reads its vertical neighbours unconditionally from always-present ghost rows, so it is branch-free in the vertical direction. At large N the kernel is compute/branch-bound (not launch-bound), so removing those branches buys 10–16 %. The "hybrid beats CUDA" headline was the hybrid's GPU slice quietly running a better kernel — a confound, not a balancing win.

5.2. Once the kernel is held constant, load balancing is a net loss

Comparing like-for-like (both on the halo kernel), hybrid-auto is below halo-only at every size — -29 % at $N = 4096$, $\approx$ break-even at $N = 16384$. The per-step coordination (ghost-row exchange + waking the worker pool + synchronising the two nodes, $\sim$0.3–1 ms/step) costs more than the 0.6 % the CPU offloads. Concretely at $N = 16384$: the DLT-optimal split saves the GPU $\sim$0.6 % of its time ($\sim$0.06 ms) by handing 94 rows to the CPU, while the coordination adds $\sim$0.6–1 ms — a clear net loss that only looks like break-even because the halo kernel's own speed-up cancels it.

5.3. The apparent "OpenCL regression" was the integrated GPU

This box previously measured OpenCL at $\sim$37 Gcells/s, on par with CUDA (both lower through the same PTX/SASS on NVIDIA). A later sweep measured 1.7 Gcells/s and was first recorded as a driver/ICD regression. It was not. With explicit device pinning (the `GOL_OCL_DEVICE` environment variable, added in `OclDeviceSelect.hpp`), the 1.7 Gcells/s figure is the Intel UHD 630 integrated GPU, which the default device selection picks because it enumerates as OpenCL platform 0. Pinned to NVIDIA, OpenCL runs at $\sim$23–28 Gcells/s, matching CUDA. So there was no regression and no reboot is needed — only a silent device choice. This reframes the exercise: the "slow OpenCL" was a second, real GPU sitting idle, which motivates the two-GPU split of $\mathcal{S}6$.

5.4. GPU run-to-run variance is $\sim$10–25 %

A single CUDA run at $N = 4096$ gave 24.9 Gcells/s vs. the best-of-5 of 31.8 — boost-clock and scheduling variance, largest at small N (per-rep CV up to $\sim$30 % at $N = 2048$, down to $\sim$1–5 % at $N = 16384$). Any margin smaller than this band — including the entire 0.6 % load-balancing prize — is unmeasurable. Report best-of-N and treat sub-10 % differences at small grids as noise.

5.5. OpenCL work-group limit at block 512

`-block 512` fails OpenCL with `CL_INVALID_WORK_GROUP_SIZE` (code −54) on this device (CUDA runs 512 fine). The sweep skips it gracefully; it is a device limit, not a regression. Block 64–128 is optimal on both engines anyway.

6. From CPU+GPU to iGPU+dGPU

The negligible-CPU verdict says the CPU is the wrong second node; the corrected OpenCL finding ($\mathcal{S}$5.3) supplies a better one — the Intel UHD 630 integrated GPU, reachable only through OpenCL. The engine was generalized from a fixed [CPU, GPU] pair to an ordered list of nodes, each a CPU pool, a CUDA discrete-GPU partition, or an OpenCL partition pinned to a chosen device. The default composition is iGPU + dGPU, with the CPU off by default; adjacent nodes exchange the same one-row ghost halo, and inter-GPU seams stage through host memory (there is no cross-vendor peer copy). Every composition (`igpu,dgpu`, `cpu,dgpu`, the three-way `cpu,igpu,dgpu`) verifies bit-for-bit against the CPU oracle in both edge modes.

6.1. The DLT model, applied

In the terms of $\mathcal{S}$11.3, the board is the divisible load $L = R{\cdot}C$ cells, node i receives a row share $part_i$, and its inverse speed is $p_i = 1/R_i$. The book's affine node cost has a computation term $p_i(part_i L + e_i)$ (Eq. 11.5) and a distribution term $l_i(a\, part_i L + b)$ (Eq. 11.7). Our ghost seam is exactly the constant b of that model: a single boundary row moved per step, independent of $part_i$ — the "boundary row of pixels" of the book's Fig. 11.5(a). The optimum (Fig. 11.9) makes both nodes finish together.

Dropping the communication constant b and the fixed overhead e_i leaves the naive closed form, which is what the auto-calibration computes:

$$part_{\mathrm{iGPU}} = \frac{R_{\mathrm{iGPU}}}{R_{\mathrm{iGPU}} + R_{\mathrm{dGPU}}} \approx \frac{2.1}{2.1 + 31.5} \approx 6.3\,\%, \tag{2}$$

with $R_{\mathrm{iGPU}} \approx 2.1$ and $R_{\mathrm{dGPU}} \approx 31.5$ Gcells/s (the pure-dGPU halo rate). This is an order of magnitude larger than the CPU's 0.6 % share, so the DLT ceiling $R_{\mathrm{iGPU}} + R_{\mathrm{dGPU}} \approx 33.6$ Gcells/s now sits ∼6.7 % above the discrete GPU alone — a prize worth chasing.

6.2. Theory versus the empirical optimum

Fig. 2 sweeps the iGPU row share directly, holding the composition at `igpu,dgpu` and measuring steady-state throughput (best of five, explicit `-fracs` so no calibration bias, long runs so the boost clocks are ramped). Each curve rises to a peak near a 5–6 % iGPU share and then falls off a cliff: past ∼7 % the slow iGPU is handed more rows than it can finish inside the dGPU's step time and becomes the bottleneck. The empirical peaks sit at 5–6 % (four

of the five grids peak at 6 %), straddling the naive 6.3 % of Eq. (2) to within the sweep's 1 % grid resolution: near the top the throughput curve is flat, so best-of-five boost variance moves the winning grid point by ±1 step. It is tempting to read the sub-6.3 % peaks as the seam and the iGPU's launch overhead lowering its effective rate — but $\mathcal{S}6.4$ measures those terms and feeds them into the reference DLT library, and the genuine correction to the optimum is under 0.05 percentage points, far too small to explain a shift to 5 %. The sub-6.3 % readings are sampling noise, not a real overhead-driven bias.

Tabla 4: iGPU+dGPU fraction sweep (best of five, steady state). "dGPU" is the pure discrete-GPU halo rate ($part_{\text{iGPU}} = 0$); "best" is the peak over the sweep at the listed iGPU share. Ceiling = $R_{\text{iGPU}} + R_{\text{dGPU}}$ with $R_{\text{iGPU}} = 2.1$. Rates in Gcells/s.

N	dGPU	best (iGPU share)	win	ceiling	% of ceiling
8192	30.43	30.77 (6 %)	+1.1 %	32.53	94.6 %
16384	31.06	32.07 (6 %)	+3.2 %	33.16	96.7 %
24576	31.58	32.73 (5 %)	+3.6 %	33.68	97.2 %
32768	31.62	33.36 (6 %)	+5.5 %	33.72	98.9 %
40960	31.54	33.17 (6 %)	+5.2 %	33.64	98.6 %

6.3. Why the win grows with the grid

The decisive observation is the scaling (Table 4, Fig. 2(B)). The seam b is a single row — a fixed C bytes per step — while the compute load $L = R \cdot C$ grows as N^2. The communication-to-computation ratio is therefore $\sim b/L \sim 1/N$ and vanishes as the grid grows, so the achievable throughput must approach the DLT ceiling $R_{\text{iGPU}} + R_{\text{dGPU}}$. That is what happens: the best static split beats pure dGPU by +1.1 % at $N = 8192$, growing monotonically to +5.5 % at $N = 32768$ (+5.2 % at 40960, within variance of the peak). As a fraction of the ceiling, the hybrid climbs from 94.6 % to 98.9 %, while pure dGPU stays pinned at ∼93.7 % — it is simply leaving the iGPU's ∼6 % on the table. At the largest grids the two-GPU hybrid captures essentially the entire theoretical bound: DLT behaving exactly as $\mathcal{S}11.3$ describes in the large-load limit, where communication is negligible and the partition delivers the sum of the nodes' rates.

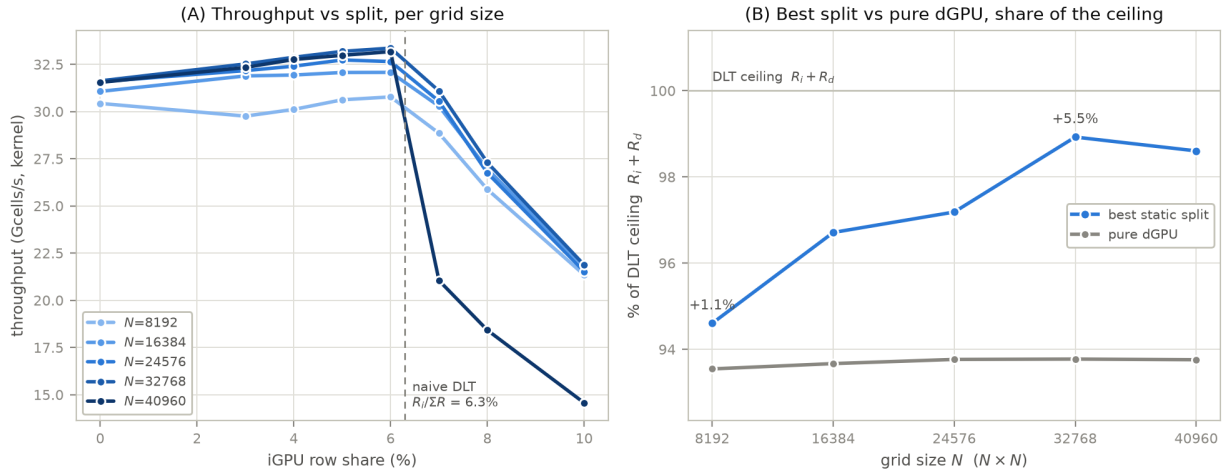

Figura 2: iGPU+dGPU fraction sweep. (A) Steady-state throughput vs the iGPU row share, one curve per grid size (best of five; darker = larger N). Each curve rises to a peak at 5–6 % then falls off a cliff as the iGPU is overloaded past the dGPU's step time — steepest at the largest grid; the peaks straddle the naive-DLT share $R_i/(R_i{+}R_d) \approx 6.3\,\%$ (dashed) within the sweep's 1 % step. (B) Best static split and pure dGPU as shares of the $R_i + R_d$ ceiling, on one axis: the hybrid climbs from 94.6 % to 98.9 % while pure dGPU stays pinned at ∼93.7 %; the gap between the two curves is the hybrid's win over pure dGPU (annotated at the ends, +1.1 % → +5.5 %), growing as the fixed one-row seam is amortized against N^2 compute.

This reverses the pessimistic reading from the CPU+GPU study and the small grids: the earlier "a few percent below pure dGPU" was measured only at $N \leq 16384$, where the seam still costs an appreciable share and the margin hid inside the ∼5–10 % boost variance. With clean measurement and large grids, a well-chosen static iGPU+dGPU split is a real, if modest, win — one whose size is governed by the divisible-load communication term, not by luck.

6.4. DLTlib cross-check: the overhead is negligible

To separate genuine divisible-load effects from measurement noise, we fed the platform into the reference library the engine is modelled on — DLTlib (Barlas, Appendix G) — and compared its optimum to ours. The platform maps to DLTlib's single-level tree as an idle load-originating root (a pure distributor) with one compute child per device of processing speed $power_i = 1/R_i$; the library's fixed-cost term $e_{0,i}$ is in load units, so a physical per-step overhead τ_i (seconds) enters as $e_{0,i} = \tau_i R_i$.

The naive split is the library's optimum.

With communication switched off ($link = 0$, $e_0 = 0$) DLTlib's `SolveImage` returns $part_i = R_i / \sum_j R_j$ to six digits — the iGPU share 6.30 % and, for the CPU+GPU pair, 0.58 %. So the engine's hand-rolled Eq. (2) is not an approximation: it *is* the $\mathcal{S}11.3$ optimum, and the

library confirms it exactly.

The measured overhead does not move it.

We then measured each device's affine per-step timing $t = \tau_i + \text{cells}/R_i$ by running it solo across board sizes (Fig. 3): $R_{\text{iGPU}} = 2.12$, $R_{\text{dGPU}} = 31.5\,\text{Gcells/s}$ (matching the sweep), with fixed overheads of only $\tau_{\text{iGPU}} \approx 23\,\mu\text{s}$ and $\tau_{\text{dGPU}} \approx 6\,\mu\text{s}$. Feeding (R_i, τ_i) back into DLTlib gives the overhead-aware optimum of Table 5: it stays within 0.05 percentage points of the naive 6.30 % at every grid size, and the correction vanishes as $1/N^2$.

Tabla 5: DLTlib optimum with the measured fixed overhead ($\tau_{\text{iGPU}} = 23\,\mu\text{s}$, $\tau_{\text{dGPU}} = 6\,\mu\text{s}$) folded in via $e_0 = \tau R$, versus the naive share and the fraction-sweep peak. The overhead correction is $\leq 0.05\,\text{pp}$ at every N.

N	naive $R_i/\Sigma R$	DLTlib + fitted e_0	empirical peak
8192	6.30 %	6.25 %	∼6 %
16384	6.30 %	6.29 %	∼6 %
24576	6.30 %	6.30 %	∼5 %
32768	6.30 %	6.30 %	∼6 %
40960	6.30 %	6.30 %	∼6 %

The reason is scale: at $N \geq 8192$ a step is milliseconds while the fixed overhead is tens of microseconds, so $\tau/t_{\text{step}} \lesssim 10^{-2}$ and the split is overhead-insensitive. The prediction is therefore a flat 6.3 %, and the empirical peaks straddle it within one sweep step (Fig. 4). The reading that the optimum "sits below 6.3 % because of the seam and launch overhead" does not survive this measurement: neither the fixed launch cost nor the one-row seam is within two orders of magnitude of what a $-1.3\,\text{pp}$ shift would require. The 5–6 % scatter is grid resolution and boost variance, not a physical bias. (As a consistency check, adding a *proportional* link cost instead moves the share the wrong way — up, toward an even split — because DLTlib's link models transfer cost proportional to the load moved, whereas the GoL seam is a fixed single row: the overhead belongs in e_0, not the link.)

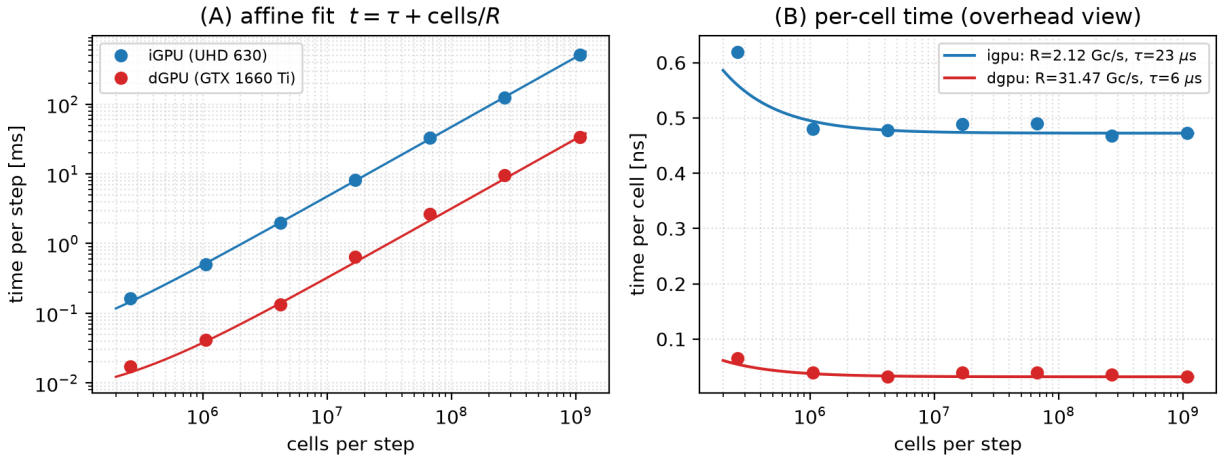

Figura 3: Per-device affine timing $t_{\text{step}} = \tau + \text{cells}/R$, each device run solo through the hybrid harness. (A) time per step vs. cells (log–log): the slope gives R. (B) time per cell: the fixed overhead τ shows only as the small-N bump (leftmost point) and is gone by $N = 1024$, pinning $\tau_{\text{iGPU}} \approx 23\,\mu$s, $\tau_{\text{dGPU}} \approx 6\,\mu$s.

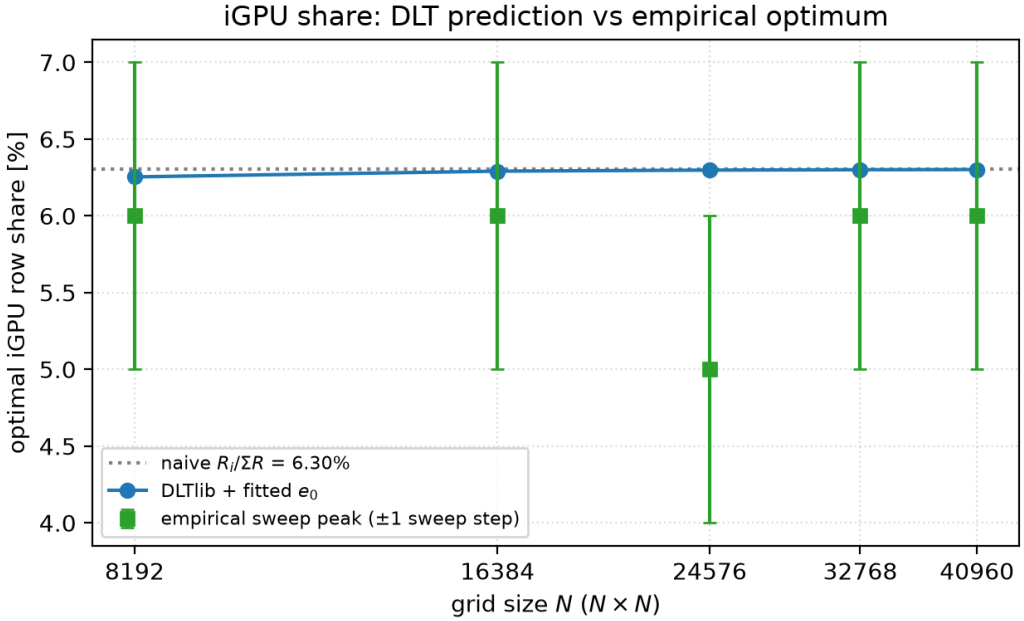

Figura 4: DLTlib optimum with fitted overhead (blue) vs. the fraction-sweep peaks (green, ±1 sweep step) and the naive $R_i/\Sigma R$ (dotted). The prediction is a flat 6.3 %; the empirical peaks straddle it within resolution — confirming the split is the naive DLT optimum and the sub-6.3 % readings are noise.

6.5. The catch: the auto-split overshoots

The win requires the correct fraction, and the default auto-calibration does not reliably find it — not because the naive Eq. (2) is wrong ($\mathcal{S}$6.4 shows it is the exact optimum) but because a short in-run calibration mis-measures the rates that feed it. It is biased by boost warm-up: the dGPU needs a long ramp to reach its 31.5 Gcells/s peak ($\mathcal{S}$6.4), so a brief calibration under-measures it and inflates the iGPU share above the true 6.3 %. Because the throughput curve past the optimum falls off a cliff while the approach is gentle, this asymmetry makes overshoot far costlier than undershoot. Lengthening calibration (`–calib-steps 50`) helps but does not fully cure it. So today the win is reachable with a hand-set `–fracs 0.05,0.95`; making the default land there — by calibrating the dGPU with a proper boost warm-up so its rate is not under-measured, or simply by damping the split toward the dGPU — is the obvious next step.

7. Conclusion: does static load balancing pay off here?

The implementation is correct and the theory is vindicated. Whether it pays off depends on the node pair and the grid size.

- Correctness. The hybrid verifies bit-for-bit against the `CpuEngine` oracle in both edge modes and across all splits; the calibrated partition matches the $\mathcal{S}$11.3 optimum to the row; the two nodes finish together. As a teaching realization of DLT on a genuinely heterogeneous CPU+GPU pair (disjoint memory, real seam communication), it does exactly what the book says.

- But this is the textbook "one node dominates" regime. With the GPU ∼160× the CPU, the DLT-optimal CPU share is 0.6 %, so the theoretical ceiling over pure GPU is ∼0.6 % — and the per-step ghost exchange + pool wakeup + sync exceed that. The hybrid ranges from −29 % (small/mid N, overhead dominated) to break-even (large N, overhead amortized). It never meaningfully beats pure GPU.

- One step up the device hierarchy, it does pay off. Replacing the CPU with the idle integrated GPU ($\mathcal{S}$6) raises the minority share to ∼6 %, and because the seam is a single fixed row its cost vanishes as $1/N$. A well-chosen static iGPU+dGPU split then beats pure dGPU by a margin that grows with the grid — up to +5.5 % at $N = 32768$, reaching 99 % of the $R_{\text{iGPU}} + R_{\text{dGPU}}$ ceiling. The prize is real here; at small N it hid inside the boost noise, and the default auto-split still overshoots it ($\mathcal{S}$6.5).

- The only speed-up in the data is a kernel-engineering side effect. The ghost-cell halo kernel, built to make domain decomposition possible, is incidentally 10–16 % faster than the original global kernel at large N because it trades vertical boundary branches for unconditional ghost-row reads.

Recommendations

1. Ship pure GPU with the halo (branch-free) kernel. That is the real lever this exercise surfaced — a free 10–16 % at large N, no CPU, no coordination. Consider promoting the halo formulation to the default global kernel (it needs only a one-row zero-pad at the board's top/bottom under bounded edges).

2. The hybrid pays off under either of two conditions: the node rates are within a small factor (a vectorized / bit-sliced CPU, a weaker / shared GPU), so the minority share is large; or the load is large enough that the fixed one-row seam is negligible against N^2 compute. The CPU+GPU pair meets neither (0.6 % share, and the grids do not go high enough); the iGPU + dGPU pair meets the second at $N \gtrsim 16384$ and wins by up to +5.5 % ($\mathcal{S}6$). Use it there with a hand-set `-fracs` near 0.05–0.06.

3. Fix the auto-calibrated split so the default `-engine hybrid` lands on the ∼5–6 % iGPU optimum instead of overshooting into the cliff: fold a communication/overhead term into the calibration, or damp the split toward the dGPU ($\mathcal{S}6.5$). There is no OpenCL pathology to fix — the slow figure was the integrated GPU ($\mathcal{S}5.3$).

The bottom line: static load balancing is the right tool for balanced heterogeneous nodes, and the implementation proves it can be done correctly and cheaply. On a GTX 1660 Ti paired with a scalar CPU stencil it has nothing to balance — the honest result is "≈ pure GPU", and the interesting performance story is the kernel, not the split.